



J A V A W E B P R O G R A M M I N G · L E C T U R E 1 2

Filters, Interceptors & Listeners

Building reactive, cross-cutting concerns in Spring Boot

Spring Boot 4

Spring Security 7

Servlet 6

Java 24

doc. dr. sc. Aleksander Radovan, prof. struč. stud.

Lecture outline

01

Filters

Servlet-level request/response interception

02

FilterChainProxy

How Spring Security plugs into the servlet chain

03

Handler Interceptors

preHandle, postHandle, afterCompletion

04

Listeners & Events

Synchronous decoupling via the application context

05

Side-by-side comparison

When to reach for which mechanism

Introduction to filters

WHAT IS A FILTER?

A servlet-level component that sits in front of every HTTP request and response

Filters belong to the **Jakarta Servlet API** (formerly `javax.servlet`) — they live in the web container (e.g. Apache Tomcat) and run **outside the Spring MVC scope**.

They decide whether to **let a request reach** the `DispatcherServlet`, modify it, or **block the outgoing response**.

Spring Security plugs into this chain via **`DelegatingFilterProxy`** — a thin servlet filter that hands control to the Spring-managed **`FilterChainProxy`**.

KEY PROPERTIES

Container-bound

Tied to the servlet container (Tomcat, Jetty)

First & last gate

Sees every request before any Spring code runs

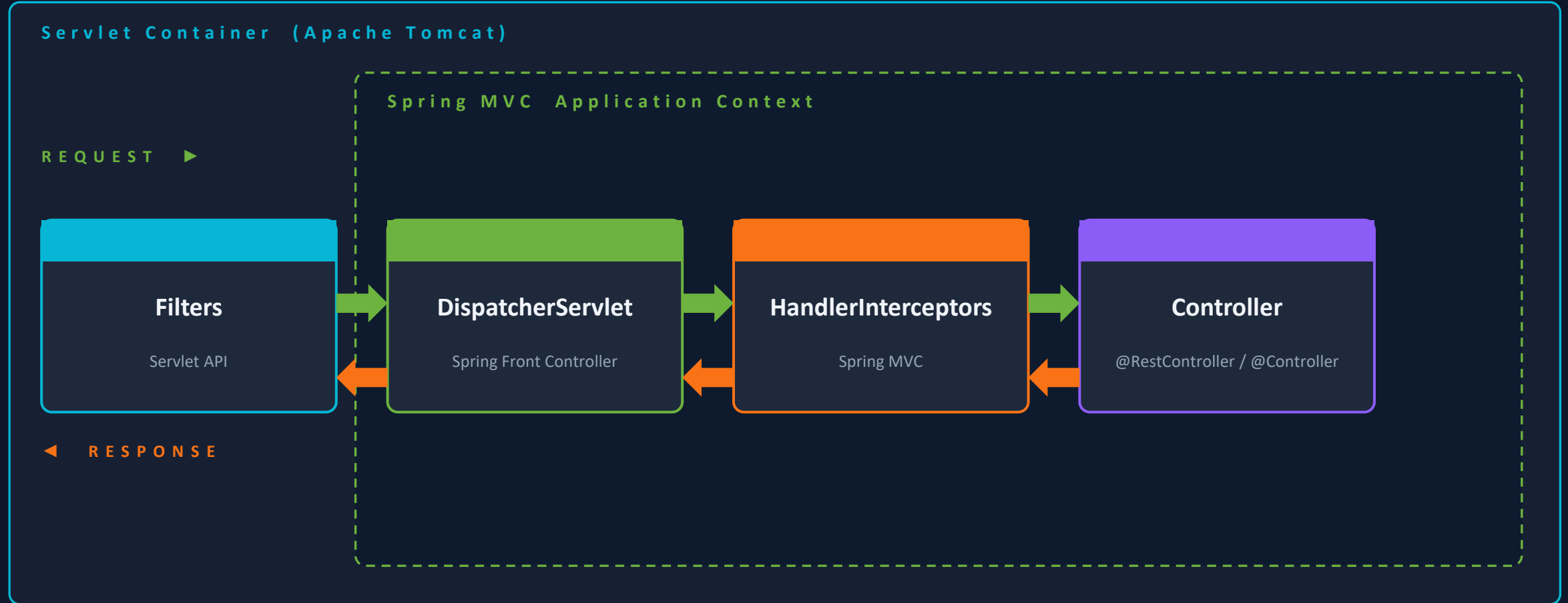
Chained

Many filters compose into a single `chain.doFilter()` pipeline

Bidirectional

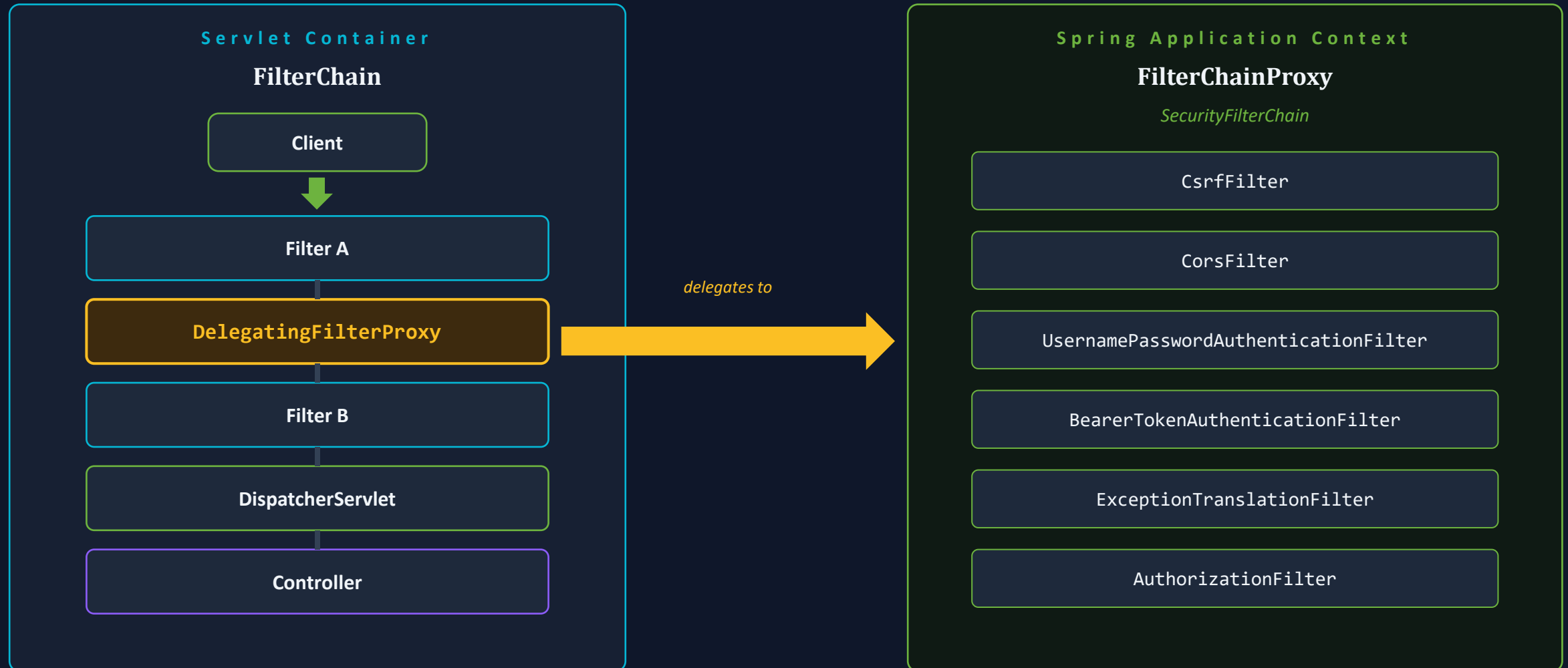
Can transform both inbound requests and outbound responses

Where filters & interceptors live



Filters run in the container — even before Spring exists. **Interceptors** run inside the Spring context, between dispatch and the handler.

FilterChainProxy: Spring Security in the chain



DelegatingFilterProxy is the bridge: a plain servlet filter registered with the container that hands control to the Spring-managed *FilterChainProxy*.

Building your first filter

A filter implements `jakarta.servlet.Filter` and overrides `doFilter()`. Calling `chain.doFilter(...)` forwards the request to the next filter — or the servlet.

`@Component`

```
public class RequestLogFilter implements Filter {

    private static final Logger log = LoggerFactory.getLogger(RequestLogFilter.class);

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
        throws IOException, ServletException {

        long started = System.currentTimeMillis();
        HttpServletRequest http = (HttpServletRequest) req;

        log.info("--> {} {} from {}",
            http.getMethod(), http.getRequestURI(), http.getRemoteAddr());

        chain.doFilter(req, res); // forward through the chain

        log.info("<-- {} {} took {} ms",
            http.getMethod(), http.getRequestURI(),
            System.currentTimeMillis() - started);
    }
}
```

Java

The most common Spring Security filters

DisableEncodeUrlFilter	Strips JSESSIONID from URL rewrites	WebAsyncManagerIntegrationFilter	Bridges SecurityContext into async requests
SecurityContextHolderFilter	Loads SecurityContext at start of request	HeaderWriterFilter	Adds security headers (XFO, CSP, HSTS)
CorsFilter	Enforces CORS policy for cross-origin calls	CsrfFilter	Validates the CSRF token for state-changing methods
LogoutFilter	Detects logout URLs and clears the session	UsernamePasswordAuthenticationFilter	Processes form-based login submissions
BearerTokenAuthenticationFilter	Reads OAuth2 / JWT Bearer tokens from headers	ExceptionTranslationFilter	Maps AccessDenied / AuthN exceptions to HTTP
AuthorizationFilter	Decides if the principal may access this URL	JwtAuthFilter (custom)	Most teams add their own JWT filter — see next slide

Order matters. Spring Security composes these into a single *SecurityFilterChain* executed strictly in declared order — register custom filters with *.addFilterBefore(...)* or *.addFilterAfter(...)*.

Custom JwtAuthFilter

Extend `OncePerRequestFilter` so the filter runs exactly once per dispatch, even on async or include / forward requests.

```
@Component
@RequiredArgsConstructor
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtService jwt;
    private final UserDetailsService users;

    @Override
    protected void doFilterInternal(HttpServletRequest req,
        HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {

        String header = req.getHeader("Authorization");
        if (header == null || !header.startsWith("Bearer ")) {
            chain.doFilter(req, res); // anonymous – let next filter decide
            return;
        }
        String token = header.substring(7);
        String username = jwt.extractUsername(token);

        if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails user = users.loadUserByUsername(username);
            if (jwt.isTokenValid(token, user)) {
                var auth = new UsernamePasswordAuthenticationToken(
                    user, null, user.getAuthorities());
                auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(req));
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}
```

Java

When do you reach for a filter?



Authentication

Decode tokens, populate
SecurityContext



Logging & tracing

Access logs, MDC, correlation IDs



Image conversion

Resize / re-encode on the fly (webp, avif)



Data compression

gzip / br response bodies before flush



Encryption

Decrypt payloads, encrypt responses



Tokenization

Replace sensitive values with surrogate
tokens



Event dispatching

Trigger async events for data-access calls



XSL / T filtering

Transform XML responses with
stylesheets

Handler Interceptors

INSIDE SPRING MVC

A Spring-managed component that wraps every controller invocation

Interceptors live inside the `org.springframework.web.servlet` package and are part of Spring MVC — not the Servlet API.

They sit between the `DispatcherServlet` and your `@Controller` — they can see the resolved handler method and the model that will be rendered.

Create one by implementing `HandlerInterceptor` and registering it via `WebMvcConfigurer#addInterceptors`.

THREE LIFECYCLE METHODS

1

`preHandle()`

Runs BEFORE the controller. Return false to stop the chain.

2

`postHandle()`

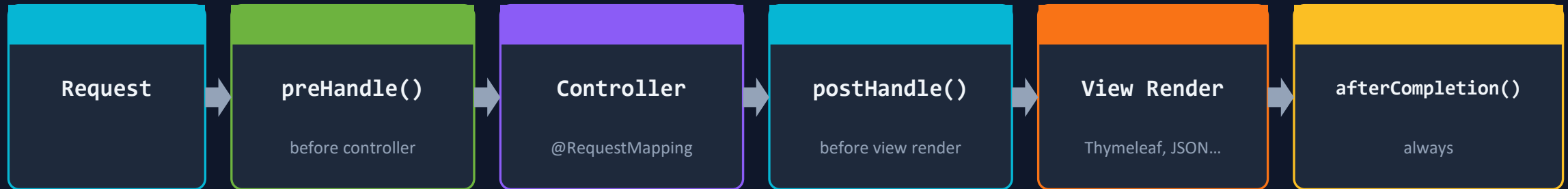
Runs AFTER the controller, BEFORE the view is rendered.

3

`afterCompletion()`

Runs AFTER the view is rendered — even when an exception was thrown.

Interceptor lifecycle in detail



preHandle()

```
boolean preHandle(req, res, handler)
```

Last chance to short-circuit. Return false to send the response without invoking the handler — perfect for security checks.

postHandle()

```
void postHandle(req, res, handler, modelAndView)
```

Gives access to the ModelAndView before view rendering — add common attributes, tweak headers, instrument timing.

afterCompletion()

```
void afterCompletion(req, res, handler, ex)
```

Always runs (even after exceptions). Ideal for releasing resources, recording metrics, flushing tracers.

LogInterceptor — full implementation

@Component

Java

```
public class LogInterceptor implements HandlerInterceptor {

    private static final Logger log = LoggerFactory.getLogger(LogInterceptor.class);

    @Override
    public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) {
        long started = System.currentTimeMillis();
        req.setAttribute("startedAt", started);
        log.info("preHandle -> {} {}", req.getMethod(), req.getRequestURI());
        return true; // false would skip the handler
    }

    @Override
    public void postHandle(HttpServletRequest req, HttpServletResponse res,
                          Object handler, ModelAndView mav) {
        log.info("postHandle -> status {} view {}",
                res.getStatus(),
                mav == null ? "(none)" : mav.getViewName());
    }

    @Override
    public void afterCompletion(HttpServletRequest req, HttpServletResponse res,
                              Object handler, Exception ex) {
        long started = (long) req.getAttribute("startedAt");
        log.info("afterCompletion -> {} ms result={}",
                System.currentTimeMillis() - started,
                (ex == null) ? "OK" : ex.getClass().getSimpleName());
    }
}
```

Interceptor use cases

Fine-grained security checks

Inspect the resolved handler method to apply per-endpoint policies that depend on Spring context (e.g. checking `@PreAuthorize` annotations programmatically).

Header & cookie manipulation

Set CORS, caching, correlation-id, or feature-flag headers on every response — using `ModelAndView` access for view metadata.

Performance instrumentation

Record latency, hits per controller, and slow-handler alerts. Pair `preHandle` / `afterCompletion` to time the entire request.

Async / saga orchestration

Trigger compensating actions, fire-and-forget events, or hand off long jobs to a `TaskExecutor` after the handler returns.

Locale & tenancy resolution

Resolve and bind locale or tenant for the rest of the request — e.g. setting `LocaleContextHolder` or a `TenantContext`.

Audit & compliance logging

Emit structured audit events with the user, action, and outcome — `afterCompletion` guarantees coverage even on errors.

Listeners & application events

THE PATTERN

Publish events. Let any number of listeners react — independently.

Events are synchronized activities inside a Spring application — fired by an **event publisher** and consumed by one or more **event listeners**.

Create events by extending **ApplicationEvent** — Spring also accepts plain POJOs since version 4.2.

Publishers use **ApplicationEventPublisher**, listeners implement **ApplicationListener<E>** or use **@EventListener**.

BUILT-IN LISTENER FLAVORS

ServletContextListener

Fires on app startup / shutdown — JNDI lookups, warm-up tasks

HttpSessionListener

Tracks session creation & destruction (idle timeouts, online users)

ServletRequestListener

Wraps every incoming request — low-level access logging

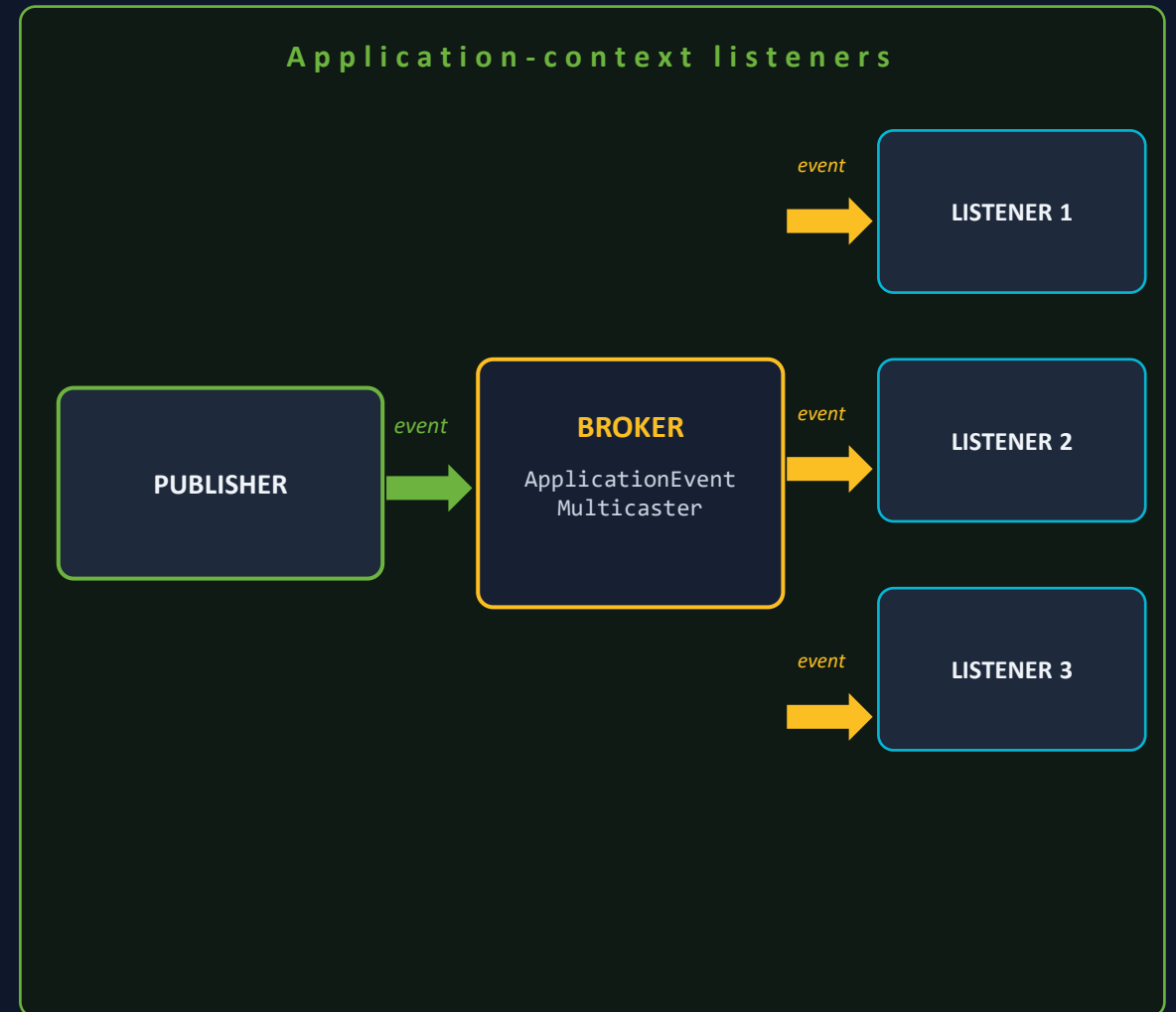
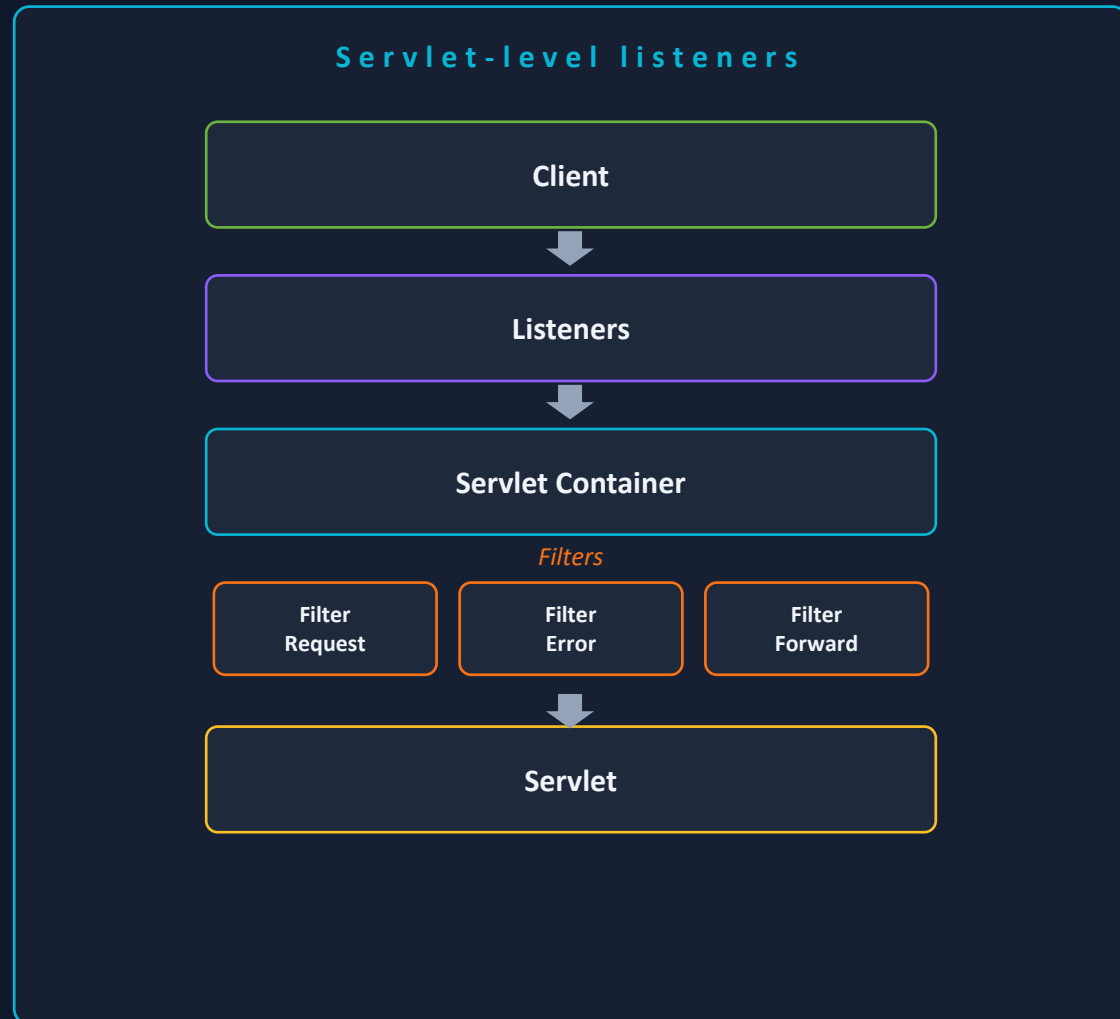
ApplicationListener<E>

Spring-managed listener for ContextRefreshed, ContextStopped, custom...

@EventListener (custom)

Annotation-driven listeners on any bean method — most modern style

Listener architecture



Common listener types

Listener / interface	Triggered by	Typical use
ServletContextListener	contextInitialized / contextDestroyed	Bootstrap, warm-up, shutdown hooks
HttpSessionListener	sessionCreated / sessionDestroyed	Track active sessions, idle-timeout cleanup
HttpSessionAttributeListener	attribute Added / Replaced / Removed	React to session-scoped state changes
ServletRequestListener	requestInitialized / requestDestroyed	Low-level per-request resource management
ApplicationListener<ContextRefreshedEvent>	Spring context fully loaded	Run once after startup — schedule warmups, integrity checks
ApplicationListener<ContextClosedEvent>	Spring context shutting down	Graceful teardown of caches, connection pools
@EventListener (any bean)	any ApplicationEvent or POJO	Modern, annotation-driven — preferred style for custom events
@TransactionalEventListener	AFTER_COMMIT / AFTER_ROLLBACK	Fire side-effects only when the DB transaction succeeds

Define a custom event

Extend `ApplicationEvent` to carry domain data — anything your listeners need to react.

```
/**
 * Published when a customer successfully places an order.
 * Listeners may send confirmation emails, update analytics,
 * trigger fulfillment, etc. — independently of each other.
 */
public class OrderPlacedEvent extends ApplicationEvent {

    private final String    orderId;
    private final String    customerEmail;
    private final BigDecimal total;

    public OrderPlacedEvent(Object source, String orderId,
                           String customerEmail, BigDecimal total) {
        super(source);
        this.orderId      = orderId;
        this.customerEmail = customerEmail;
        this.total         = total;
    }

    public String    getOrderId()      { return orderId;      }
    public String    getCustomerEmail() { return customerEmail; }
    public BigDecimal getTotal()       { return total;       }
}
```

Java

Publish the event

Inject `ApplicationEventPublisher` — every Spring bean can do this — and call `publishEvent(...)`.

```
@Service
@RequiredArgsConstructor
public class OrderService {

    private final OrderRepository orders;
    private final ApplicationEventPublisher publisher;

    @Transactional
    public Order placeOrder(NewOrderRequest req) {
        Order saved = orders.save(Order.from(req));

        // Fire and forget — every registered listener will react
        publisher.publishEvent(new OrderPlacedEvent(
            this,
            saved.getId(),
            saved.getCustomerEmail(),
            saved.getTotal()
        ));

        return saved;
    }
}
```

Java

Two ways to listen

You can implement the listener as an interface, or as a method annotated with `@EventListener`:

Java

```
// 1) Interface-based listener
@Component
public class OrderEmailListener
    implements ApplicationListener<OrderPlacedEvent> {

    private final MailService mail;

    public OrderEmailListener(MailService mail) {
        this.mail = mail;
    }

    @Override
    public void onApplicationEvent(OrderPlacedEvent ev) {
        mail.sendConfirmation(
            ev.getCustomerEmail(),
            ev.getOrderId());
    }
}
```

Java

```
// 2) Annotation-driven listener
@Component
@RequiredArgsConstructor
public class OrderAnalyticsListener {

    private final AnalyticsClient analytics;

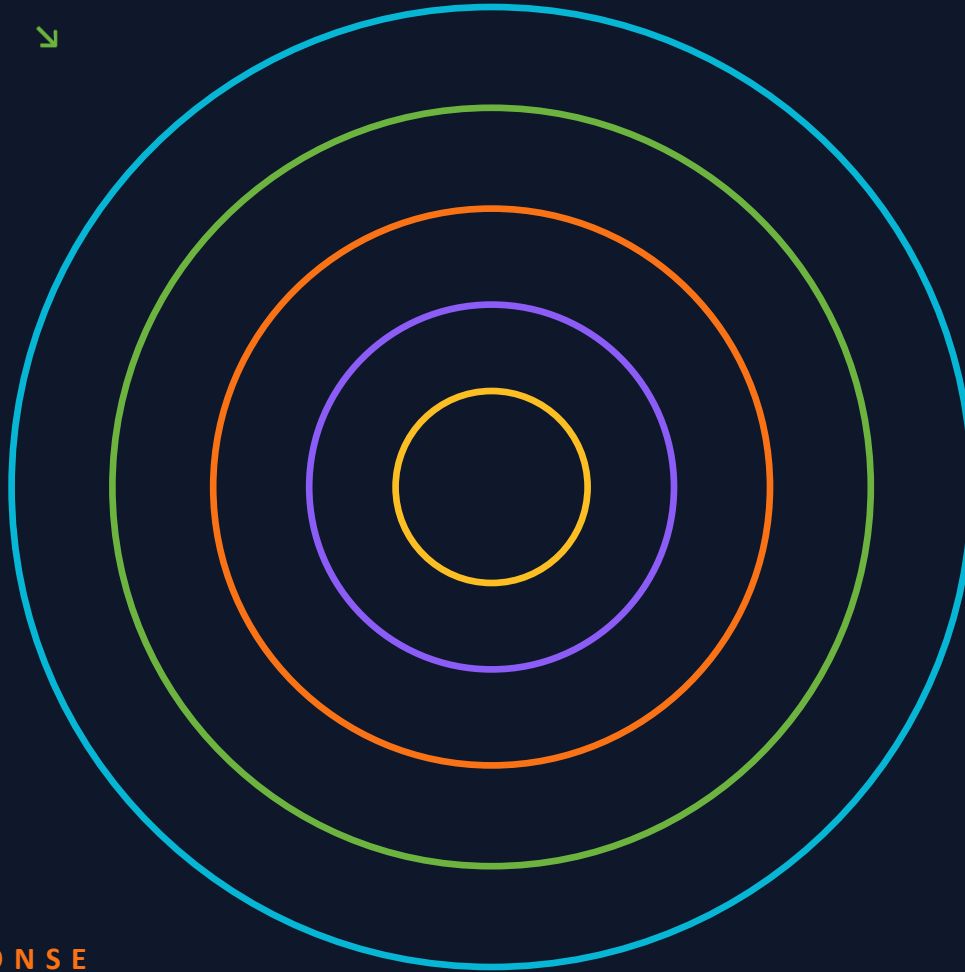
    @EventListener
    public void onOrderPlaced(OrderPlacedEvent ev) {
        analytics.track("order_placed", Map.of(
            "id",    ev.getOrderId(),
            "value", ev.getTotal()
        ));
    }
}
```

Filters · Interceptors · Listeners

	Filter	Interceptor	Listener
Package / origin	jakarta.servlet (Servlet API)	org.springframework.web.servlet	org.springframework.context
Defined in	Web container config — outside the Spring context	Inside the Spring application context	Inside the Spring application context
Runs around	Servlet calls — before & after dispatch	Handler methods & view rendering	Published events (sync or async)
Bound to	The servlet container (Tomcat, Jetty...)	Spring MVC (web layer)	Just the Spring context — works in any app type
Sees the handler?	No — only raw HTTP	Yes — has the resolved handler method	N/A — sees the event payload
Typical use	Cross-cutting HTTP concerns: auth, gzip, logging	Spring-aware pre/post logic & instrumentation	Decoupled domain notifications & side-effects

Hierarchical circles

REQUEST ↘



↗ RESPONSE

Tomcat / Servlet Container
Lives outside Spring — the gateway

Servlet & Filters
Filters run here, before dispatch

DispatcherServlet
Spring's front controller

Interceptor
Wraps controller invocation

Controller
Your `@RestController` / `@Controller`

Each ring runs inside the next — the further in, the more Spring-aware the context.

Registering everything in Spring Boot

Java

```
// Filter:  adds itself via @Component
//          or FilterRegistrationBean
@Configuration
public class WebFiltersConfig {

    @Bean
    FilterRegistrationBean<RequestLogFilter> log(
        RequestLogFilter f) {
        var reg = new FilterRegistrationBean<>(f);
        reg.addUrlPatterns("/api/*");
        reg.setOrder(1);
        return reg;
    }
}

// Security filter – plug into the chain:
@Bean
SecurityFilterChain security(HttpSecurity http,
    JwtAuthFilter jwt) throws Exception {

    return http
        .csrf(CsrfConfigurer::disable)
        .addFilterBefore(jwt,
            UsernamePasswordAuthenticationFilter.class)
        .build();
}
```

Java

```
// Interceptor: register through WebMvcConfigurer
@Configuration
public class WebMvcConfig implements WebMvcConfigurer {

    private final LogInterceptor logInterceptor;

    public WebMvcConfig(LogInterceptor i) {
        this.logInterceptor = i;
    }

    @Override
    public void addInterceptors(InterceptorRegistry reg) {
        reg.addInterceptor(logInterceptor)
            .addPathPatterns("/api/**")
            .excludePathPatterns("/api/health");
    }
}

// Listener: just be a @Component – Spring scans
//            the @EventListener methods at startup.
// (no extra wiring needed!)
```

References & further reading

- | | | |
|----|-------------------------------|---|
| 01 | Spring Security Architecture | docs.spring.io/spring-security/reference/servlet/architecture.html |
| 02 | HandlerInterceptor vs. Filter | baeldung.com/spring-mvc-handlerinterceptor-vs-filter |
| 03 | Spring Events Guide | baeldung.com/spring-events |
| 04 | @TransactionalEventListener | docs.spring.io/spring-framework/reference/data-access/transaction/event.html |
| 05 | OncePerRequestFilter Javadoc | docs.spring.io/spring-framework/docs/current/javadoc-api/org.springframework.web.filter.OncePerRequestFilter.html |
| 06 | Spring Boot reference docs | docs.spring.io/spring-boot/index.html |



Thank you!

Java Web Programming